

CUDA PROGRAMMING FOR NVIDIA H100S

WGMMMA 简介

Prateek Shukla · 简体中文版

hopper 中 **mma** 的 4 项重要创新

除了拥有更快的张量核心之外，还有以下重要创新：

1. 张量核心操作现在完全异步。数学指令现在是非阻塞的。这使得多个操作可以同时`in-flight`，意味着张量核心会有更长的活跃时间。2. 我们已经从用单个 warp 执行张量核心操作，过渡到使用 `warpgroup`，后者允许使用大得多的 tile。3. 能够用异步硬件直接为 tile 从共享内存/寄存器取数。在 `ampere` 上，数据到达寄存器之前线程无法发射 `mma.sync` 操作。在 `wgmma` 中我们可以跳过加载寄存器，直接发射数学操作。4. 对 `fp8` 和稀疏性的专用硬件支持。

范式转变：**warp** 与 **warp group**

在 Hopper 上，我们已经告别了长达十年之久的以 warp 作为执行单元的强调，把它改成了 warp group。当你调用 wgmma 时，你是在把 4 个 warp 融合成一个在张量核心上运作的单一计算实体。这最大化了张量核心的利用率，也避免了把多条 mma.sync 操作流水线化的麻烦。全部 4 个 warp 一起发射 wgmma 指令。warp 调度器检查所有 warp 是否在 PC 上于该指令处收敛，然后调度器将它们融合，并把数学命令派发给张量核心。命令发送到张量核心之后，线程就继续去执行下一条指令。

WGMMA 流水线

从 tile 异步加载数据，必要时预加载下一个缓冲区。

如果要把 A tile 加载进寄存器，就使用 `ldmatrix` 和一堆复杂的寻址；加载 B tile 则使用 `wgmma` 描述符。

否则，只需为 A 和 B tile 创建描述符即可。

发射 `wgmma` 操作。

从寄存器收集结果。发射 `wgmma.fence.sync.align` 使对寄存器的写入可见，再做些复杂的地址计算把数据搬到共享内存。

wgmma.fence.sync.aligned

`wgmma.fence` 是面向 `wgmma.mma_async` 的寄存器排序屏障，不是完成屏障。它的职责是：在后续的 `wgmma.mma_async` 使用那些相同寄存器之前，让先前的 `warpgroup` 寄存器访问以正确的顺序变得可见。

两种主要情况下需要它：`warpgroup` 中第一条 `wgmma.mma_async` 之前；以及任何时候某线程访问过寄存器、而后续的 `wgmma.mma_async` 将把这些寄存器重新用作累加器或 A 片段输入。

它不做的事情是：为 `wgmma.mma_async` 所消费的矩阵描述符/数据的共享内存写入排序。为此，PTX 文档要求使用异步 proxy 栅栏（例如 `fence.proxy.async`），让先前的共享内存写入相对后来的 `wgmma.mma_async` 读取有序。

wgmma.mma_async

这是让张量核心执行运算的核心异步指令。

A 直接从共享内存/寄存器读取。

B 直接从共享内存读取。

C、D 驻留在寄存器中。

整个操作看起来像这样：`wgmma.mma_async.sync.aligned.shape.dtype.bf16.bf16 d, a-desc, b-desc, scale-d, imm-scale-a, imm-scale-b, imm-trans-a, imm-trans-b;`

.sync 与 .aligned

.sync —— 这是 SM 范围的屏障。它强制要求 warpgroup 中所有参与线程（通常 128 条线程，即 4 个 warp）都必须到达这条指令，之后任何线程才能继续。

.aligned —— 这个限定符断言 warpgroup 中的所有线程都在该指令的内存地址（程序计数器，PC）处收敛（以锁步方式执行），并且 warp group 内部没有线程分支发散。

指令限定符

Shape —— 你的 tile 的形状，我们用 $m64$ 、 $n\{\}$ 、 $k\{\}$ 来表示。

$dtypeD$, $dtypeA$, $dtypeB$ —— D、A、B tensor 的数据类型。

m64nXkY

对所有 wgmma 指令，M 维度固定为 64。

K 维度（内积维度）严格由输入矩阵 A 和 B 的精度决定。

N 维度最灵活。它决定处理矩阵 B（和 C）的多少列。

N 必须是底层内存分配块大小的倍数（取决于类型，通常是 8 或 16 的倍数）。

有效的 N 取值范围从 8 到 256。

固定的 m

硬件设计为每条线程持有 4 个寄存器。这意味着每个 warpgroup 可以持有 $128 * 4 = 512$ 个寄存器。

这意味着如果我们使用最小的 tile $m = 64, n = 8$ ，就得到 $64 * 8 = 512$ 个元素，恰好是一个 warpgroup 的容量。

硬件把来自 A 的数据物理地映射到张量核心输入。在 $M = 64$ 时，硬件拥有一张完美、静态的映射。这在运行时不需要任何决策逻辑。

如果让 m 可配置，你就需要在寄存器与数学单元之间放一个巨大的交叉开关（Crossbar Switch，一种复杂的多路选择器），根据你的请求动态重路由数据，这非常昂贵。

操作数位置

矩阵 A —— 寄存器/共享内存

矩阵 B —— 共享内存

矩阵 C/D —— 仅寄存器

scale_d

只是指定我们应当进行累加 ($D = A \times B + C$)

还是覆写 ($D = A \times B$) 操作。

Scale_d = 1 —— 累加

Scale_d = 0 —— 覆写

scale_a/b

这些是把 A/B 的元素按 1 或 -1 缩放的缩放因子。

寄存器操作数 **d**

这些是将要存储输出的寄存器。

在 PTX 指令字符串中，操作数 **d** 是第一个操作数。它必须声明为用花括号 **{}** 括起来的寄存器向量（元组）。

输出操作数的数量可以这样计算——

输出中的元素总数 / 线程总数

A 的位置

A 的 tile 可以驻留在共享内存或寄存器中。因此有两种方式告诉 wgmma A tile 驻留在哪里。

a. 寄存器：如果 tile 驻留在寄存器中，我们就把寄存器传给 wgmma 指令。b. 共享内存：我们传入 wgmma 描述符。

A 放在寄存器中

当你在复用数据时，这是好办法，因为从共享内存加载 A 和 B 会使 bank 压力加倍，多次写入共享内存代价很高，所以从寄存器加载更好。

从共享内存搬数据很贵。从寄存器搬数据很便宜。从寄存器加载让可复用的数据待在能被更快访问的地方。

寄存器用的数据来自共享内存，ldmatrix 指令接收未 swizzle 的地址并把数据加载进寄存器。

如果你不复用数据，那么从寄存器加载 A 就没有意义，因为存在指令开销和寄存器压力。

把数据加载进寄存器的机制

每条线程都提供用于把数据加载进寄存器的指针，ldmatrix 指令用这些数据填充目标线程的寄存器。

仅适用于 blackwell ——

ldmatrix

在 WGMMA 流水线中，它是用矩阵 A 的数据填充寄存器的主要机制。

与标准的 ld.shared（加载线性数据）不同，ldmatrix 以不透明的寄存器模式加载数据，这种模式被设计成与张量核心的输入通道在物理上对齐。

指向数据的线程有时并不是最终持有数据的线程。

.m8n8 是从共享内存加载进 warp 寄存器的矩阵 tile 的几何形状。

限定符 .x1、.x2 和 .x4 决定向量化宽度以及每条指令加载的矩阵片段数量。

x{1, 2, 4}

.x{1, 2, 4} 告诉我们能为计算搬进寄存器多少个 8×8 的核心矩阵。

.x4 —— 意味着每条线程加载 4 个寄存器，或加载 4 个 8×8 维度的核心矩阵。最常用，因为它通过从共享内存向寄存器加载 $16 * 16 * 4 = 1024$ 字节的数据来打满寄存器带宽。

.x2 —— 加载 512 字节数据，即每条线程加载 2 个寄存器。一般用于加载较小的 tile。

.x1 —— 每条线程 1 个寄存器，主要用于边界处理和其他工作。

一个 **16x16 tile** 如何从共享内存加载进寄存器

整个操作分两个阶段工作：

地址阶段：谁提供指针？（“收集”）

寄存器阶段：谁持有结果？（“目的地”）

使用 x4 时，我们处理的是 4 个 8x8 tile 区段。

“Split-Warp” 策略

我们不线性地计算地址。我们把 warp 拆成两条“竖条”（左/右），因为 ldmatrix 一次加载 8 列的块。

左半部分（第 0-7 列）：由前 16 条线程（lane 0-15）控制。

右半部分（第 8-15 列）：由接下来的 16 条线程（lane 16-31）控制。

阶段 **1**：地址职责（哪条线程提供指针）

在这个阶段，硬件查看每条线程中的地址寄存器来决定从共享内存的哪里读取。warp 被拆成 4 组，每组 8 条线程。

* @param tile_row_offset 该 tile 在更大矩阵中的全局行偏移

- @param tile_col_offset 该 tile 在更大矩阵中的全局列偏移

阶段 2：寄存器职责（谁持有数据）

数据被抓取后，ldmatrix 把它分发到整个 warp，使其为张量核心数学做好准备。分发模式是按 4 条线程一组做行条带（Row-Striped）分布。对任意 8x8 矩阵（M0、M1、M2 或 M3），行的分布如下：第 0 行给线程 0-3；第 1 行给线程 4-7；第 2 行给线程 8-11；.....第 7 行给线程 28-31。

CUDA PROGRAMMING FOR NVIDIA H100S

单个核心矩阵的布局

Prateek Shukla · 简体中文版

地址计算

tile 以 swizzle 后的布局存放在共享内存中（这样选择是为了避免 bank 冲突），因此一个“逻辑行”的字节并不是按物理地址顺序连续存放的。每条线程从自己的逻辑坐标出发（例如“我需要第 r 行、第 c 个列块”），然后用 swizzle 映射（通常是基于 XOR 的重映射）为该片段计算物理共享内存地址： $\text{smem_addr} = \text{base} + (\text{row_offset} \wedge \text{swizzle_mask}) + \text{col_offset}$ 。这个算出的地址就是该线程提供给 ldmatrix 的指针。由于指针经过了 swizzle 映射，访问会落到不同的 bank，warp 因此只会有极小甚至零 bank 冲突。ldmatrix 每条线程加载 16 字节，并以片段化布局对其重排。

地址位分解

要理解 swizzle 公式如何工作，我们需要理解地址布局。

位 [0-1]：4 字节字内的字节偏移。与 bank 选择无关。

位 [2-6]：bank 索引。这 5 位决定数据落入 32 个 bank (0-31) 中的哪一个。

具体来说，位 [4-6] 控制 bank 索引的高 3 位。

位 [7-9]：行索引。由于 pitch 是 128 字节 (2^7)，当你移动到下一行时，位 7 是第一个发生变化的位。

我们希望 bank (由位 4-6 控制) 根据行 (由位 7-9 控制) 而变化。

swizzle 后的地址计算

$\text{Physical_Address} = (\text{Base_Address} + \text{Linear_Offset}) \wedge \text{Swizzle_Mask};$

其中：

./ 128 bytes = 2^7 。我们取位 [7,8,9]，把它们移到位置 [4,5,6]。

`uint32_t mask = ((linear_offset .> 7) & 0x7) .< 4;`

./ 最终指针

`uint32_t smem_ptr = (base_ptr + linear_offset) ^ mask;`

打包

NVIDIA GPU 架构没有 16 位寄存器；它使用 32 位寄存器。因此，bf16 数据驻留在寄存器中时必须总是成对打包。

容器：一个 .b32（32 位）寄存器装两个 bf16 元素。

布局：

位 0-15（LO）：元素 N（偶数索引）

位 16-31（HI）：元素 N+1（奇数索引）

移动这类数据时，你通常使用 .b32 类型指令。计算时则使用 .bf16x2。

ldmatrix 打包

如果你使用 `ldmatrix.sync.aligned.m8n8.x1.b16`，每条线程收到一个 32 位寄存器。

这个寄存器包含按上文描述打包的两个 bf16 元素。

如果你使用 `.x2` 或 `.x4`，你会得到 2 或 4 个寄存器，每个都装着一对打包好的 bf16。

共享内存中的地址必须 16 字节（128 位）对齐。

使用 `.trans` 可以把行主序数据当作列主序（或反过来）直接加载进寄存器，无需手动洗牌。

.sync.aligned 与 .trans

.sync : 它充当一个迷你屏障。硬件需要保证 T16 已经准备好让自己的寄存器被 T0 请求的数据覆写。如果 T16 正忙于别的事情，硬件写入方就会破坏 T16 的状态。

.aligned : 所有线程必须一起执行它，这样硬件才知道 warp 已就绪。

.trans : 它告诉硬件是否在加载数据的同时转置矩阵。

由于一条线程的寄存器对其他线程不可见，整条指令由硬件在内部完成。

它们如何接入 **wgmma** 指令

当 wgmma 从特定线程（比如 Warp 0 的 Thread 0）读取寄存器 {ra0, ra1, ra2, ra3} 时，它期望这些寄存器包含矩阵的特定片段。

关键是，wgmma 被硬连线为期望 ldmatrix 产出的那种精确布局。

张量核心硬件解开这些寄存器的连线，并把它们重新映射到物理的点积单元。

指令：wgmma... {acc}, {ra0...ra3}, desc_b ...

硬件从 Warp 0 抓取 {ra0...ra3}。它知道这是第 0-15 行以及来自 warpgroup 其他部分的数据，并用来自 B 的数据执行 wgmma。

wgmma 描述符

这是一个 64 位描述符，包含所存数据、步长和 swizzle 布局的信息。

描述符打包了 5 个不同的字段：地址、LBO、SBO、矩阵基偏移（matrix base offset）和 swizzle 布局。

蓝图

一个单独的 64 位寄存器编码了基地址、K 步长、M/N 步长、矩阵基偏移和 swizzle 模式。这种紧凑结构为 tensor 操作提供了高效的内存访问模式。

每个步长都预先除以 16 以装进 14 位字段（因为共享内存指针是 16 字节对齐的，最后 4 位没有用处）。硬件在运行时乘回 16，确保 16 字节对齐并降低地址延迟，这对高性能张量核心操作至关重要。

基地址必须 16 字节对齐。这种对齐确保预处理过的步长能被硬件正确解释，维持高效的内存访问并避免性能损失。

共享内存基地址

基地址是描述符的前 14 位。它就是我们要加载的那个 tile 的共享内存地址。

我们从地址中提取这 14 位，然后把它们用作描述符的 14 位。

Leading byte offset (LBO)

沿操作数“leading”方向相邻的两个核心矩阵列在共享内存中的字节距离。LBO 占用位 16-29。要编码 K 步长，把想要的字节步长除以 16，掩码到 14 位，然后左移 16 位。K-major swizzle 布局不使用 LBO，硬件假定其为 1。对于 MN-major， $LBO = \text{从前}(\text{swizzle-byte-size}/16) \text{ 行到下一组}(\text{swizzle-byte-size}/16) \text{ 行的偏移}$ 。对于 128B swizzle， $\text{swizzle-byte-size}/16 = 128/16 = 8$ ，所以在这种归一化布局中就是“跳 8 行”。

Stride Byte Offset (SBO)

它是 WGMMA 用来 (与 LBO 一起) 在共享内存中导航矩阵操作数的另一个步长。SBO 是在 SMEM 中沿另一个 (非 leading) 方向跳过一个“核心矩阵块”的字节距离。

SBO 占用位 32-45。用同样的方式编码 M/N 步长：

对于 K-major swizzle 布局，SBO 是从前 8 行到下一组 8 行的偏移。

对于 MN-major swizzle 布局，SBO 是从前 8 列到下一组 8 列的偏移。

矩阵基偏移 (**matrix base offset**)

当 SMEM 被 swizzle (例如 128B swizzle) 时，从逻辑地址到物理 SMEM bank/行的映射使用一种重复的排列模式。对于 128B swizzle 模式，该模式每 128B 重复一次。

如果你的矩阵起始地址恰好落在该 swizzle 的“模式起点”边界上，那么 `base_offset = 0`。否则，你必须告诉硬件你是从重复区域内的哪个 128B 块开始的。这就是基偏移所编码的内容。

Swizzle 模式位

如果共享内存中的数据被 swizzle 了，那么我们需要把 swizzle 后的布局应用到 tensor 上才能正确读取数据。

如果我们没有正确设置 swizzle 布局，wgmma 单元就会把被打乱的数据当线性数据读出来。

我们有 4 种 swizzle 模式，位 61-63 控制 swizzle 模式。00 表示无 swizzle，01 表示 128B，10 表示 64B，11 表示 32B。

CUDA PROGRAMMING FOR NVIDIA H100S

完整的描述符函数

Prateek Shukla · 简体中文版

wgmma.mma_async 操作做什么

它把数据从你的共享内存/寄存器异步搬到张量核心去执行运算。消费者 warp group 中的所有线程都在调用 `wgmma.mma_async`。

张量核心开始对数据做计算。然后结果会依据 `warpId`、`laneId` 写回消费者 `warpgroup` 的寄存器。它的开销很低——硬件是在把数据倾倒入最近的内存。

硬件想把结果写到物理上最靠近负责那一块的数学单元的线程。所以线程被交错成小块，每条线程持有的数据在内存中不连续。

一条线程会在输出中拿到所有那些寄存器

如果我们使用 m64n256k16 的 wgmma，那么每条线程的累加器中有 128 个元素。每一条 WGMMA 指令都由 warpgroup 中的每一条线程发射！

对 warpgroup 中的 128 条线程：如果累加器是 16 位，你需要 16 条带 .x4 的 stmatrix 指令；如果是 32 位累加器，则需要 32 条 stmatrix。

对于 flashattention 3 这类 kernel，如果用寄存器加载 A tile，就要小心 softmax GEMM 流水线带来的寄存器压力。

imm-scale-{a,b} —— “取负” **A** 的选项，这意味着 **A** 变成 **-A**

它可以取两个值：1 或 -1，没有其他值，因为这个变换只需要翻转一个位而不是多个位。

Imm-trans-{a,b}—— 这是转置 tensor 的选项。它可以取 0 或 1。它们以位的形式存储，因为指令字符串中剩下的位不足以存放一个完整的 16 位浮点数。

CUDA PROGRAMMING FOR NVIDIA H100S

再见



Prateek Shukla · 简体中文版